Why It Helps in Broadcasting

If you have a 2D array A of shape (3, 3):

```
A = np.array([[1,2,3],
              [4,5,6],
              [7,8,9]])
```

- Row-wise mean:

```
row_mean = np.mean(A, axis=1)
print(row_mean)     # [2. 5. 8.]
print(row_mean.shape) # (3,)
```

- Compare A < row_mean ❌ fails because shapes (3,3) vs (3,) don't align correctly.
- Reshape mean to column:

```
row_mean = row_mean.reshape(-1, 1)
```

- Now row_mean.shape = (3,1)
- Broadcasting works:

```
[[1<2 2<2 3<2]
 [4<5 5<5 6<5]
 [7<8 8<8 9<8]]
```

✅ Correct element-wise comparison **row by row**.

🔹 **Key Takeaway**

- -1 → NumPy automatically figures out how many rows you need
- 1 → make 1 column
- Useful for **broadcasting along rows** or columns
-

✅ Question 5 (Advanced Usage)

```
A = np.array([1, 2, 2, 3, 3, 3, 4, 4, 4, 4])
```

1. Get unique values.
2. Get counts of each value.
3. Find the most frequent element WITHOUT using Python loops.

```
print(np.unique(A, return_counts=True))
max_count_index = np.argmax(A)
print(max_count_index)
print(A[max_count_index])
```